МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

ОТЧЕТ
по курсовой работе

Тема Разработка трехмерной сцены «Ваза с цветком» с использованием библиотеки OpenGL

Обучающегося группы И934Б (группа) Коломейчук Д.К. (Фамилия и инициалы)

Направление подготовки / специальность 09.03.01 (индекс) Информатика и вычислительная техника (полное наименование направления подготовки / специальности)

Направленность образовательной программы Автоматизированные системы обработки информации и управления (профиль / специализация / магистерская программа)

Дисциплина (модуль) Компьютерная геометрия и графика

Руководитель: _____ (подпись)

к.пед.н., доцент (ученая степень, ученое звание) Снижко Е.А. (Фамилия ИО)

Оценка: _____

« » _____ 20 ___ г.

Обучающийся: _____ (подпись)

Коломейчук Д.К. (Фамилия ИО)

« » _____ 20 ___ г.

САНКТ-ПЕТЕРБУРГ
2025 г.

РЕФЕРАТ

Пояснительная записка 18 с., 5 рис., 1 прил.

OPENGL, ТЕКСТУРА, ОБЪЕКТ, ШЕЙДЕР, КООРДИНАТЫ, ВЕКТОР, ИСТОЧНИКИ СВЕТА

Цель курсовой работы – разработка трехмерной сцены «Ваза с цветком».

В ходе выполнения курсовой работы была создана трёхмерная сцена, на объекты которой были добавлены текстуры, настроены источники света, реализована анимация, а также обеспечено управление объектом и камерой с помощью клавиатуры.

Сцена разработана в IDE MicrosoftVisualStudio 2022 на языке C++ с использованием графической библиотеки OpenGL.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 Постановка задачи 5
2 Описание разработанной программы 6
2.1 Моделирование сцены и отдельных объектов 6
2.2 Разработка шейдеров и наложение текстур 6
2.3 Анимация и управление 9
2.4 Освещение и материалы объектов 10
3 Использованные мультимедийные ресурсы и сторонние библиотеки 12
4 Демонстрация работы 14
ЗАКЛЮЧЕНИЕ 16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 17
ПРИЛОЖЕНИЕ А Исходные файлы курсовой работы 18

ВВЕДЕНИЕ

Данная курсовая работа посвящена разработке сцены «Ваза с цветком», предназначенного для визуализации трёхмерной сцены с использованием современных технологий компьютерной графики. Выбор данной темы обусловлен актуальностью применения API OpenGL для создания реалистичных трёхмерных сцен, а также возможностью продемонстрировать ключевые принципы рендеринга: работу с геометрией, текстурами, освещением, камерой и анимацией.

Целью курсовой работы является разработка трехмерной сцены «Ваза с цветком».

Для достижения поставленной цели необходимо решить следующие задачи:

– создать объекты сцены «Ваза с цветком» с использованием графических возможностей библиотеки OpenGL в соответствии с реальной моделью;

– реализовать наложение текстур на объекты;

– разработать алгоритмы анимации перемещения объектов;

– реализовать освещение объектов с помощью средств библиотеки OpenGL;

– разработать алгоритмы управления и взаимодействия со сценой, ее объектами и камерой;

– продемонстрировать работоспособность программы.

1 Постановка задачи

Целью работы является создание трёхмерной сцены «Ваза с цветком», предназначенной для визуализации интерактивной композиции с использованием технологий компьютерной графики на базе OpenGL. Сцена будет демонстрировать ключевые принципы рендеринга загрузку геометрических моделей из внешних файлов формата OBJ, наложение текстур для повышения визуальной достоверности.

Сцена должна состоять из помещения, в центре которого будет расположен деревянный стол. На столе будет расположена глиняная ваза с плавно покачивающимся цветком. На полу под столом будет расположен ковер.

Все объекты необходимо реализовать с соответствующими текстурами.

Для создания реалистичного восприятия использовать освещение. Для каждого объекта индивидуально задать параметры материалов их коэффициенты блеска, силы зеркального отражения. Реализовать динамическую анимацию покачивающегося цветка. Осуществить управление сценой с помощью клавиатуры и мыши. Управление камерой будет включать в себя вращение вокруг сцены по орбите и масштабирование, а управление вазой будет происходить перемещением по горизонтальной поверхности стола и вращением вокруг вертикальной оси.

2 Описание разработанной программы

2.1 Моделирование сцены и отдельных объектов

Все модели объектов загружены из внешних файлов в формате Wavefront OBJ с использованием библиотеки TinyOBJLoader. В коде это реализовано функцией loadOBJ, которая формирует массив вершин с позициями, нормалями и текстурными координатами. Переменная attrib содержит общие атрибуты модели и включает в себя вершины (vertices), нормали (normals), текстурные координаты (texcoords), materials это материалы, связанные с моделью, например, цвета, текстуры. Реализация функции загрузки представлена ниже.

```
bool loadOBJ(const char* path, SimpleMesh& outMesh) {
    tinyobj::attrib_t attrib;
    std::vector<tinyobj::shape_t> shapes;
    std::vector<tinyobj::material_t> materials;
    std::string warn, err;
    bool ret = tinyobj::LoadObj(&attrib, &shapes, &materials,
&warn, &err, path);
    if (!err.empty()) {
        std::cerr << "Error loading OBJ: " << err << std::endl;
        return false;
    }
    if (!ret) return false;
```

Для задания объекта инициализируются Vertex Array Object (VAO) и Vertex Buffer Object (VBO). Вершинные данные, включающие позицию, нормаль и текстурные координаты это 8 значений типа float, загружаются в буфер, а затем описываются атрибуты вершин их компоненты, смещения и шаг чтобы OpenGL корректно интерпретировал структуру данных при рендеринге. Реализация этой настройки представлена во фрагменте ниже.

```
auto setupVAO = [&](int i, const SimpleMesh& mesh) {
    glBindVertexArray(VAOs[i]);
```

```
    glBindBuffer(GL_ARRAY_BUFFER, VBOs[i]);
    glBufferData(GL_ARRAY_BUFFER, mesh.vertexCount * 8 *
sizeof(GLfloat), mesh.vertices, GL_STATIC_DRAW);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 *
sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8 *
sizeof(float), (void*)(3 * sizeof(float)));
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8 *
sizeof(float), (void*)(6 * sizeof(float)));
    glEnableVertexAttribArray(2);
    }
```

Стены и пол создаются программно с помощью функции createQuad(). Эта функция строит прямоугольник из двух треугольников, задавая для каждой вершины не только координаты, но и нормаль всегда направленную вперёд, вдоль оси Z и текстурные координаты.

Ваза, стол, цветок и ковёр это внешние 3D-модели в формате OBJ, которые загружаются из файлов vase.obj, table.obj, flower.obj, carpet.obj, соответственно, с помощью функции loadOBJ().

Камера реализована с помощью функции glm::lookAt. Она расположена на орбите вокруг сцены и всегда направлена на вазу. Код ниже реализует орбитальную камеру, которая вращается вокруг вазы на фиксированной высоте, всегда держа её в центре кадра.

```
glm::vec3 camPos(
     cameraRadius * sin(cameraAngle),
     2.0f,
     cameraRadius * cos(cameraAngle)
 );
 glm::vec3 lookAtPoint(vaseX, 0.6f, 0.0f);
```

```
 glm::mat4 view = glm::lookAt(camPos, lookAtPoint, glm::vec3(0.0f, 1.0f, 0.0f));
 glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
 glUniform3fv(viewPosLoc, 1, &camPos[0]);
```

2.2 Разработка шейдеров и наложение текстур

Вершинный шейдер преобразует входные вершины, нормали и UV-координаты и передаёт их во фрагментный шейдер вычисляя итоговую позицию вершины на экране с помощью модели, вида и проекции. Фрагмент представлен ниже.

```
#version 330 core
in vec3 aPos; in vec3 aNormal; in vec2 aTexCoord;
uniform mat4 model, view, projection;
out vec3 FragPos, Normal; out vec2 TexCoord;
void main() {
    FragPos = vec3(model * vec4(aPos, 1.0));
    Normal = mat3(transpose(inverse(model))) * aNormal;
    TexCoord = aTexCoord;
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

Фрагментный шейдер вычисляет цвет пикселя по модели Phong складывая фоновое освещение (ambient слабый постоянный свет, имитирующий многократно отражённый свет в помещении. Здесь он составляет 10% от цвета текстуры), рассеянное (diffuse основной поток света, зависящий от угла между нормалью поверхности и направлением) и зеркальное (specular блик, видимый, когда камера находится в зоне отражения) освещение.

Текстуры загружаются из файлов (JPEG/PNG) с помощью библиотеки stb_image. После загрузки пикселей они передаются в OpenGL, и для каждой текстуры настраиваются параметры отображения. Вызывается функция

минмап glGenerateMipmap(), которая создаёт уменьшенные копии текстуры, что устраняет шум при отдалении а так же для гладкости изображения используется фильтрация при уменьшении трилинейная (GL_LINEAR_MIPMAP_LINEAR) создает плавный переход, при увеличении билинейная (GL_LINEAR) обеспечивает сглаживание пикселей. Функция GL_REPEAT позволяет бесшовно закрасить большие поверхности маленькой текстурой.

Таким образом, текстуры имитируют разные материалы:

– wood_texture.jpg — деревянный стол (глубокий коричневый, глянцевый);

– vase_diffuse.jpg — матовая керамика;

– flower_diffuse.png — цветок;

– carpet_texture.jpg — тканевый ковёр;

– wall_texture.jpg и floor_texture.jpg — обои и каменный пол.

2.3 Анимация и управление

Анимация реализована только для цветка: его стебель плавно покачивается с синусоидальной амплитудой, зависящей от времени:

```
float sway = sin(glfwGetTime() * 2.0f) * 3.0f;
flowerModel = glm::rotate(flowerModel, glm::radians(sway), glm::vec3(0.0f, 1.0f, 0.0f));
```

Управление камерой по орбите осуществляется при помощи кнопок A/D, колесиком мыши реализовано приближение к вазе. Для управления вазой нужно нажать на кнопку C и перемещать по столу с помощью A/D, а также вазу можно поворачивать относительно своей оси с помощью кнопок Q/E. Реализация кнопок в коде представлена ниже.

```
if (moveCamera) {
    if (keys[GLFW_KEY_A]) cameraTargetAngle += CAMERA_ROT_SPEED * dt;
    if (keys[GLFW_KEY_D]) cameraTargetAngle -= CAMERA_ROT_SPEED * dt;
```

```
}
else {
    if (keys[GLFW_KEY_A]) vaseTargetX -= VASE_MOVE_SPEED * dt;
    if (keys[GLFW_KEY_D]) vaseTargetX += VASE_MOVE_SPEED * dt;
    if     (keys[GLFW_KEY_Q])     vaseTargetRotation     +=
VASE_ROT_SPEED * dt;
    if     (keys[GLFW_KEY_E])     vaseTargetRotation     -=
VASE_ROT_SPEED * dt;
}
vaseTargetX = std::max(-1.2f, std::min(1.2f, vaseTargetX));
```

2.4 Освещение и материалы объектов

В сцене используется единственный направленный источник света, расположенный в точке (-4.0, 5.0, 3.0). Что имитирует верхнее боковое освещение, как от окна или лампы.

```
glm::vec3 lightPos(-4.0f, 5.0f, 3.0f);
glm::vec3 lightColor(1.0f, 1.0f, 0.9f);
glUniform3fv(lightPosLoc, 1, glm::value_ptr(lightPos));
glUniform3fv(lightColorLoc, 1, glm::value_ptr(lightColor));
```

Для каждого объекта задаются индивидуальные параметры материала, передаваемые в шейдер через uniform-переменные. Чтобы достичь бликов на поверхностях задается shininessLoc, со степенью глянца тем выше значение, тем более сфокусированный и резкие блик, соответствующий глянцевым поверхностям таким как стол. Чем ниже значение, тем шире и рассеяннее блик, соответствующий матовым поверхностям таким как ваза. Определение интенсивности зеркальности объекта задается с помощью specularStrengthLoc.

```
// Стол — глянцевый
glUniform1f(shininessLoc, 128.0f);
glUniform1f(specularStrengthLoc, 0.85f);
// Ваза — матовая керамика
glUniform1f(shininessLoc, 8.0f);
```

```
glUniform1f(specularStrengthLoc, 0.012f);
// Стены — матовые
glUniform1f(shininessLoc, 2.0f);
glUniform1f(specularStrengthLoc, 0.02f);
```

3 Использованные мультимедийные ресурсы и сторонние библиотеки

В проекте были использованы следующие сторонние библиотеки:

- stb_image.h - загрузка изображений текстур в форматах JPG, PNG без внешних зависимостей;
- TinyOBJLoader - загрузка 3D-моделей из файлов формата Wavefront OBJ.

Все трёхмерные модели (vase.obj, table.obj, flower.obj, carpet.obj) и текстуры (vase_diffuse.jpg, wood_texture.jpg, flower_diffuse.png, carpet_texture.jpg, floor_texture.jpg, wall_texture.jpg) были загружены из открытых интернет-источников, таких как Free3D. На рисунке 1 представлен пример поиска для объекта стола.

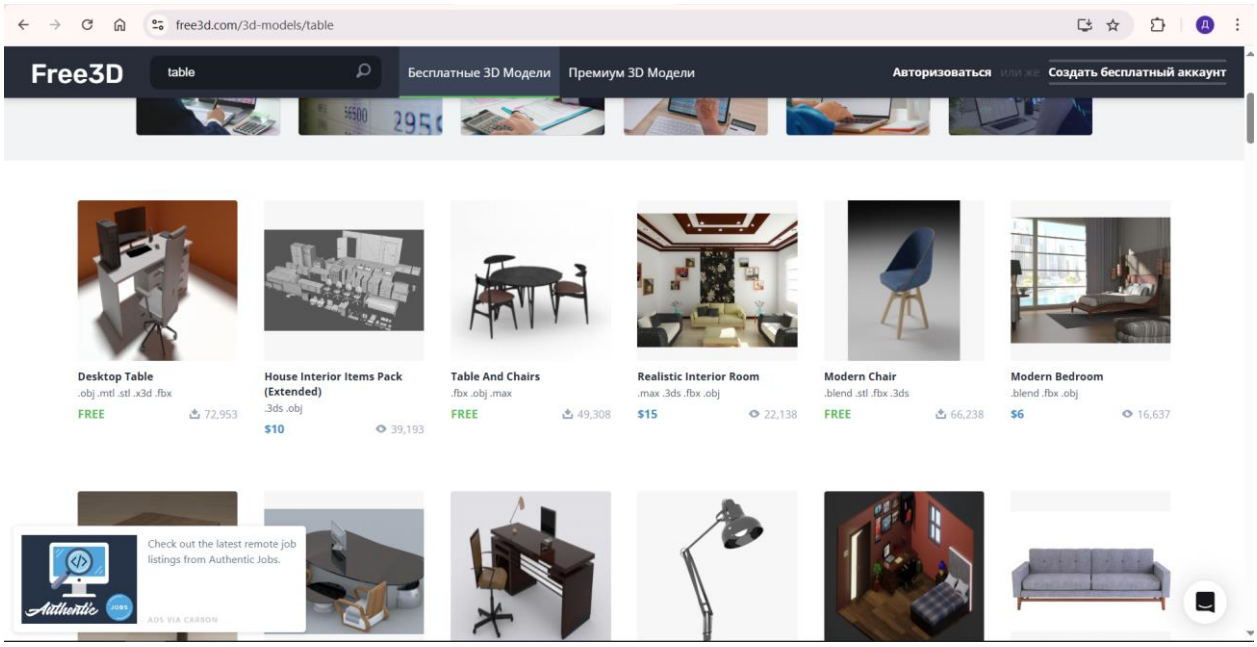


Рисунок 1 – Поиск объекта

Текстуры также взяты с открытых источников, такого как ambientcgCG, предоставляющих материалы под лицензией CC0 (Public Domain). На рисунке 2 представлен пример поиска текстуры дерева на вышеупомянутом ресурсе.

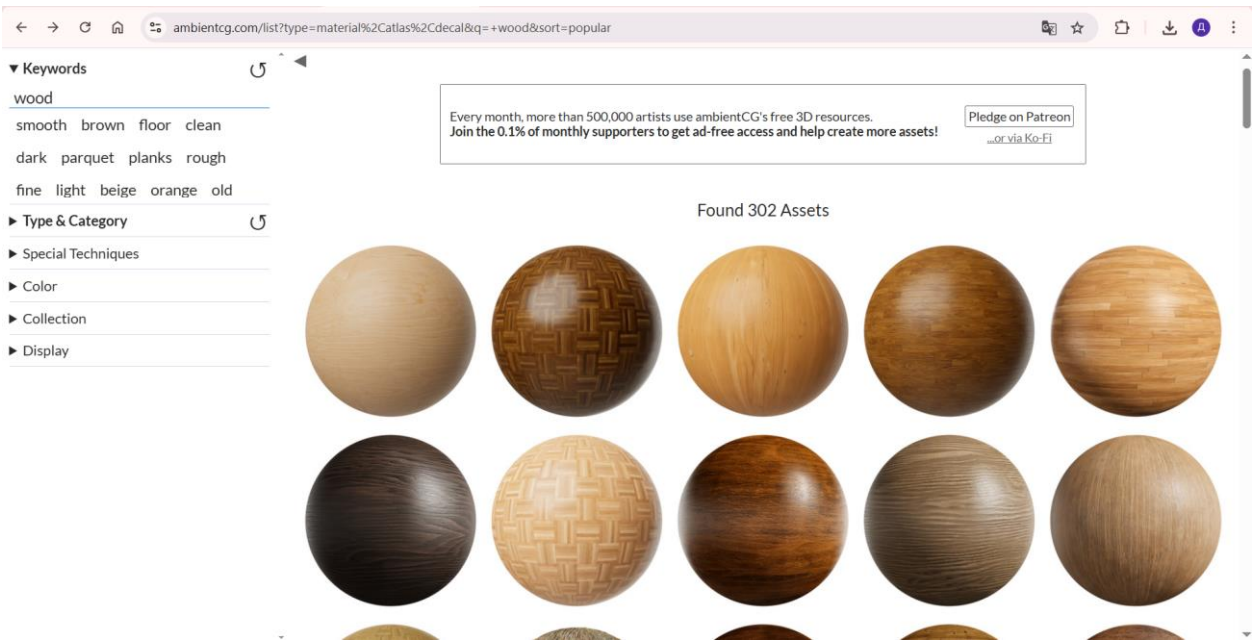


Рисунок 2 - Поиск текстуры дерева

4 Демонстрация работы

На рисунке 3 представлено общее изображение сцены и изменение положения цветка с анимацией покачивания.



Рисунок 3 – Покачивание цветка

На рисунке 4 видно, что при попадании освещения на поверхность материалы имеют разные отражающие характеристики, а также показано управление камерой и возможность масштабирования.



Рисунок 4 – Различие поверхностей объектов

На рисунке 5 показано управление объектом ваза, перемещение по столу и поворот вокруг своей оси.

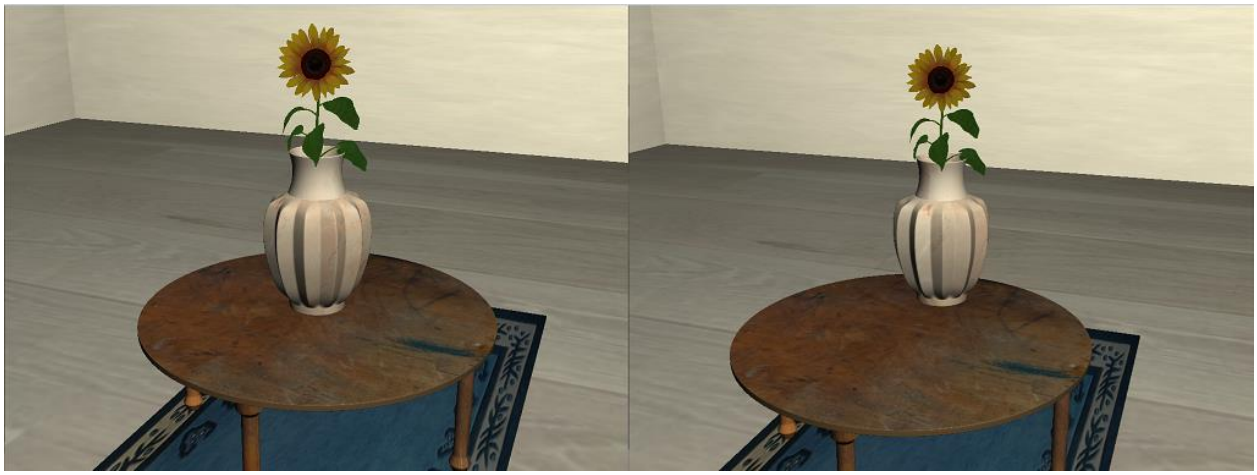


Рисунок 5 – Перемещение вазы

ЗАКЛЮЧЕНИЕ

Сцена «Ваза с цветком» включает в себя следующие объекты: керамическую вазу, цветок, деревянный стол, ковёр, пол и четыре стены. Все модели загружены из внешних файлов формата Wavefront OBJ с использованием библиотеки TinyOBJLoader, а плоские элементы сгенерированы программно.

На все объекты сцены были наложены соответствующие текстуры - керамики, древесины, ткани, стен и напольного покрытия. Загрузка текстур выполнена с помощью библиотеки stb_image, а корректное отображение обеспечено за счёт передачи текстурных координат в шейдеры.

Для освещения сцены реализована Phong-модель, включающая фоновую, диффузную и зеркальную компоненты отражения. Использован направленный источник света, с тёплым белым цветом. Параметры материалов индивидуально подобраны для каждого объекта, что позволило добиться реалистичного восприятия поверхностей.

Динамическая анимация реализована для цветка, который плавно покачивается из стороны в сторону.

Управление сценой обеспечено через клавиатуру и мышь: клавишей C пользователь переключается между режимами — управление камерой и управление вазой. Движение и вращение реализованы с применением аффинных преобразований.

Курсовая работа выполнена в IDE Microsoft Visual Studio 2022 на языке C++ с использованием графической библиотеки OpenGL. Исходные тексты разработанной сцены представлены в приложении А.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ресурс для скачивания текстур. – URL: https://ambientcg.com/?spm=a2ty_o01.29997173.0.0.19aa51710ZvJho (дата обращения 02.12.2025).

2. Библиотека для загрузки объектов .obj. – URL: https://github.com/tinyobjloader/tinyobjloader (дата обращения 03.12.2025).

3. Библиотека загрузки текстур. – URL: https://github.com/planetack/stb_image (дата обращения 04.12.2025).

4. Составные объекты. – URL: https://free3d.com/ (дата обращения 05.12.2025).

5. Использование OpenGL. – URL: https://ps-group.github.io/cg/mmz05 (дата обращения 06.12.2025).

ПРИЛОЖЕНИЕ А

Исходные файлы курсовой работы

Исходные файлы приложения доступны по ссылке: https://github.com/darakolomejcuk/course-work.